Recitation 9 – Sliding Window Protocols and TCP

Daniel Karalnik and Liam Tal

Computer Networks

Table of Contents

1. Sliding Window Protocols
2. TCP Fundamentals
3. TCP Flow Control
4. TCP Connection Management
5. TCP Congestion Control
6. Practice Questions

Today

- Sliding Window Protocols (GBN, SR)
- Minimum Sequence Numbers
- TCP Fundamentals (Header, ACKs, Retransmission)
- TCP Flow Control
- TCP Connection Management
- TCP Congestion Control (Slow Start, Congestion Avoidance)
- TCP Tahoe vs TCP Reno
- Practice Questions

Why Sliding Window?

The Problem with Stop-and-Wait

In Stop-and-Wait, sender waits for ACK after each packet:

- Very low utilization, especially with high RTT
- $U_{sender} = \frac{L/R}{RTT+L/R}$

Solution: Pipelining

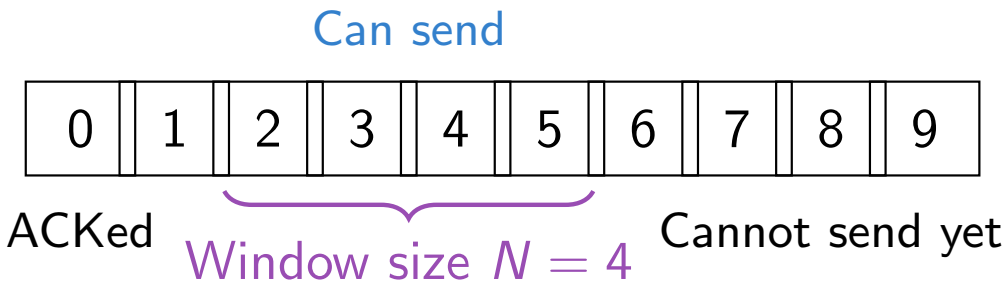
Allow multiple unacknowledged packets "in flight":

- Sender can send N packets without waiting for ACKs
- Need buffering at sender and/or receiver
- Need larger sequence number range

Go-Back-N (GBN)

Go-Back-N Protocol

- Sender can have up to N unacknowledged packets in pipeline
- Receiver sends **cumulative ACKs** (ACK n = received all up to n)
- Receiver discards out-of-order packets (no buffering)
- On timeout: sender retransmits **all** unacknowledged packets



Selective Repeat (SR)

Selective Repeat Protocol

- Sender can have up to N unacknowledged packets in pipeline
- Receiver sends **individual ACKs** for each packet
- Receiver **buffers** out-of-order packets
- On timeout: sender retransmits **only** the missing packet

Key Differences from GBN:

Feature	**GBN**	**SR**
ACK type	Cumulative	Individual
Receiver buffering	No	Yes
Retransmission	All from lost	Only lost packet
Complexity	Lower	Higher

Minimum Sequence Numbers

How Many Sequence Numbers Do We Need?

To avoid ambiguity between old and new packets:

Go-Back-N:

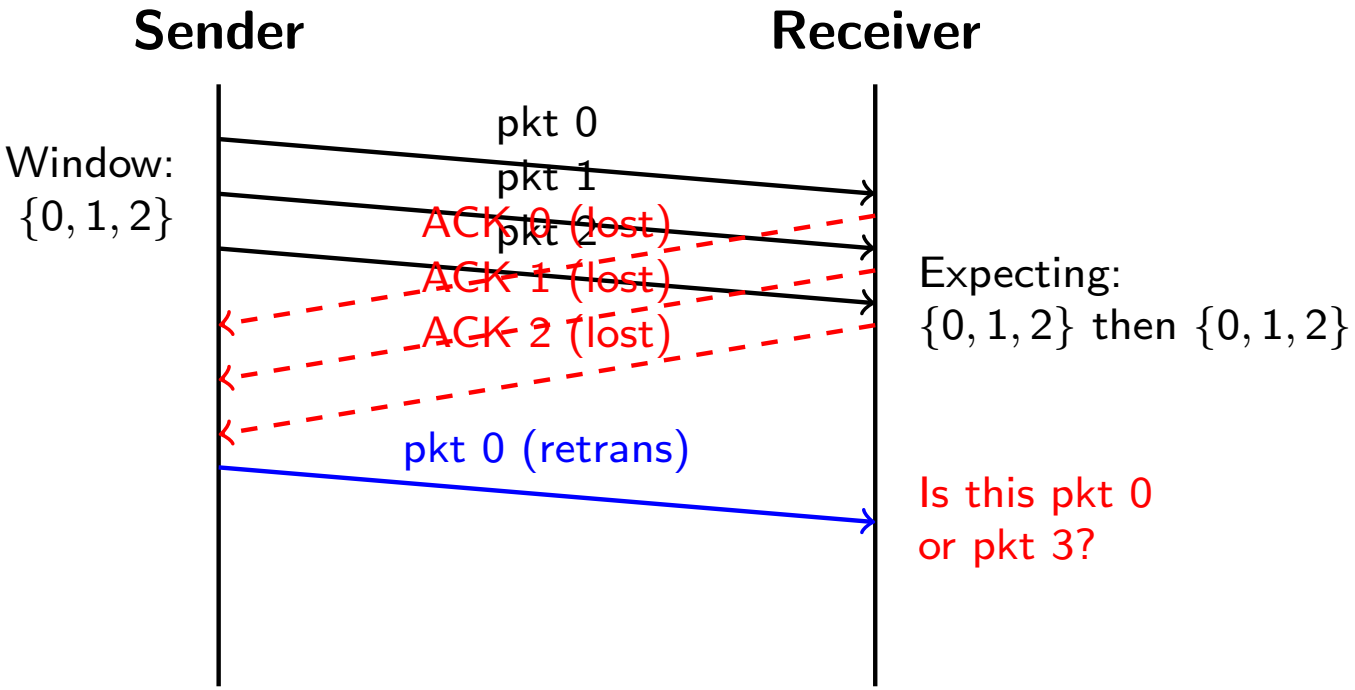
- Need at least $N + 1$ sequence numbers
- Receiver only expects next in-order packet
- Example: Window $N = 4 \Rightarrow$ need seq # $\{0, 1, 2, 3, 4\}$

Selective Repeat:

- Need at least $2N$ sequence numbers
- Both sender and receiver have windows of size N
- Windows must not overlap after wraparound
- Example: Window $N = 4 \Rightarrow$ need seq # $\{0, 1, ..., 7\}$

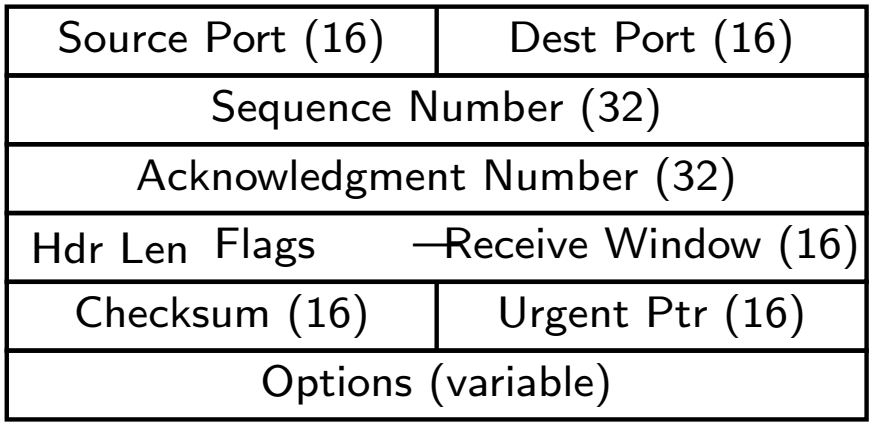
SR Sequence Number Problem

Why does SR need $2N$ sequence numbers?



With only N seq numbers, receiver cannot distinguish retransmission from new packet!

TCP Header Overview



Key Fields for This Recitation:

- **Sequence #**: Byte offset of first data byte in segment
- **ACK #**: Next byte expected (cumulative ACK)
- **Receive Window**: Flow control (rwnd)
- **Flags**: SYN, ACK, FIN, RST

TCP Sequence Numbers

Sequence Numbers in TCP

- Sequence numbers count **bytes**, not packets
- Initial Sequence Number (ISN) chosen randomly
- Each segment's seq # = byte offset of first data byte

Example:

- ISN = 1000, sending 3 segments of 500 bytes each:
  - Segment 1: seq = 1000 (bytes 1000-1499)
  - Segment 2: seq = 1500 (bytes 1500-1999)
  - Segment 3: seq = 2000 (bytes 2000-2499)

ACK Number: Next byte expected. If received bytes 1000-1499, send ACK = 1500.

TCP ACK Mechanisms

TCP uses cumulative ACKs with additional mechanisms:

1. **Cumulative ACK**: ACK n means "received all bytes up to $n-1$"
2. **Delayed ACK**: Wait up to 500ms for more data before ACKing
   - Reduces ACK traffic
   - ACK every 2nd segment or after timeout
3. **Duplicate ACKs**: Re-send ACK when out-of-order segment arrives
   - Signals to sender that a gap exists
   - Used for fast retransmit

Fast Retransmit

After receiving **3 duplicate ACKs**, retransmit the missing segment immediately (don't wait for timeout).

GBN vs SR vs TCP

Feature	GBN	SR	TCP
ACK type	Cumulative	Individual	Cumulative
Receiver buffer	No	Yes	Yes
Retransmit	All unACKed	Only lost	Only lost*
Duplicate ACKs	No	No	Yes
Fast retransmit	No	No	Yes

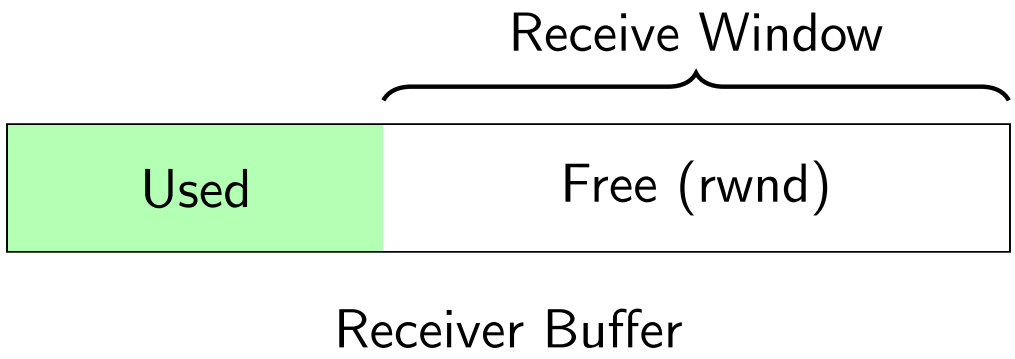
TCP is a hybrid: Uses cumulative ACKs like GBN, but buffers out-of-order segments like SR. Fast retransmit provides quick loss recovery.

*TCP retransmits only the oldest unACKed segment on timeout or fast retransmit

Flow Control with Receive Window

Flow Control

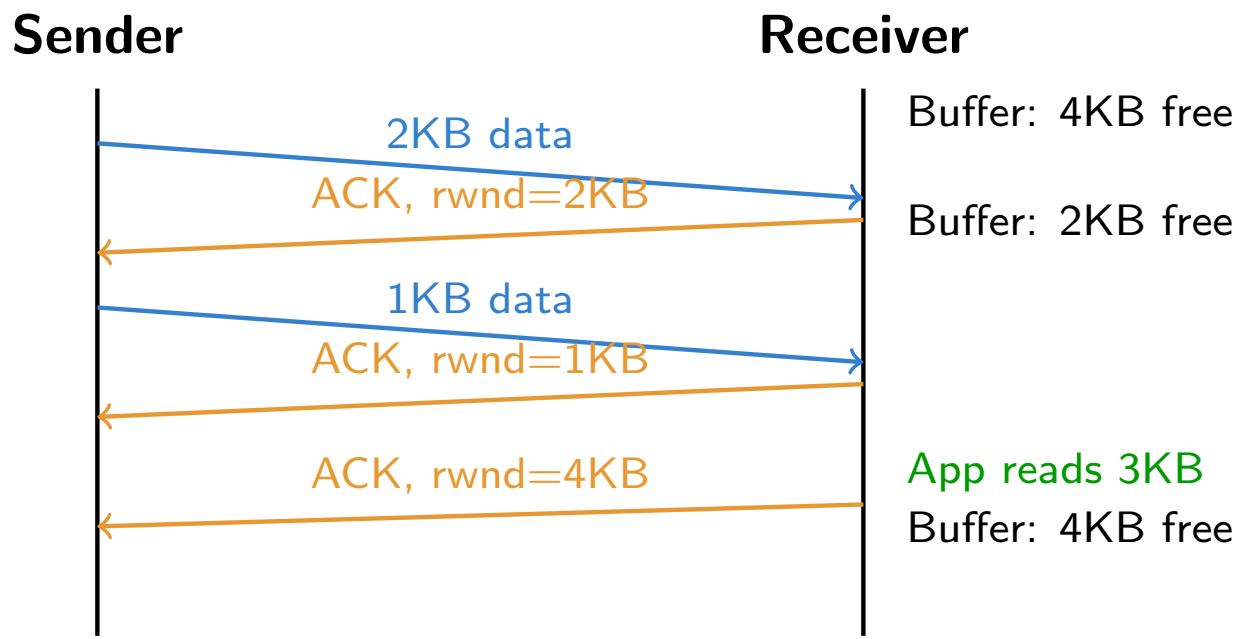
Prevents sender from overwhelming receiver's buffer.



How it works:

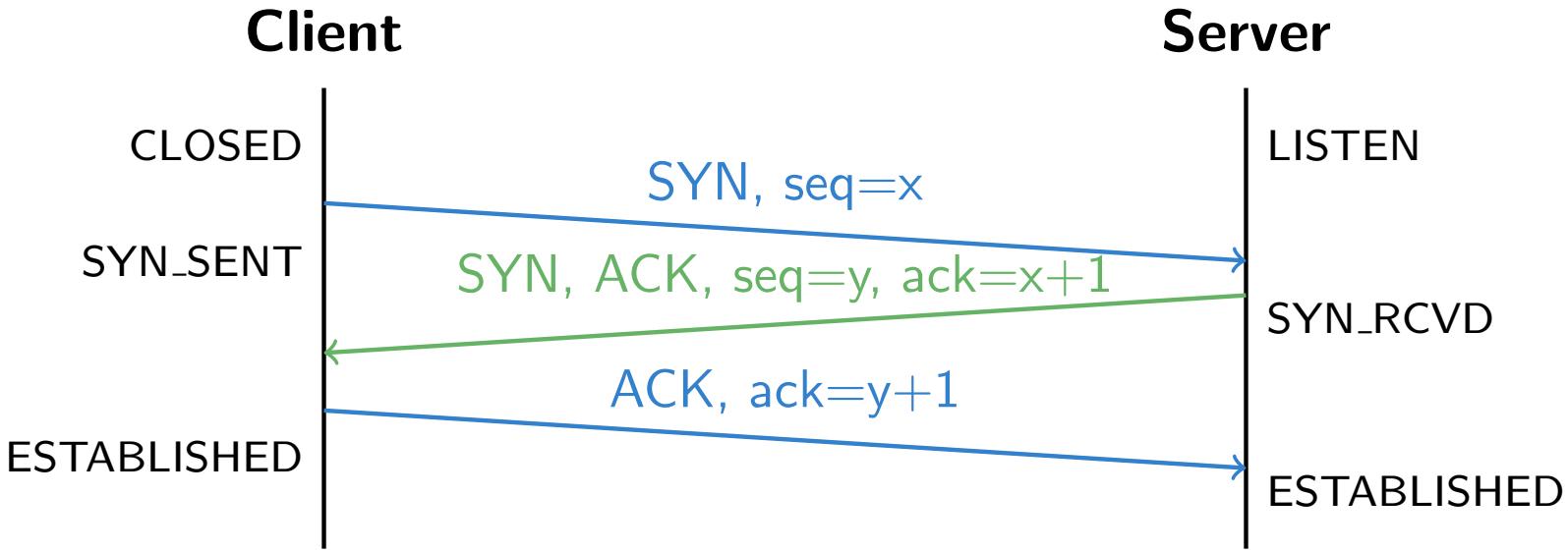
- Receiver advertises **rwnd** (receive window) in each ACK
- rwnd = free buffer space
- Sender limits unACKed data: LastByteSent − LastByteAcked ≤ rwnd

Flow Control Example



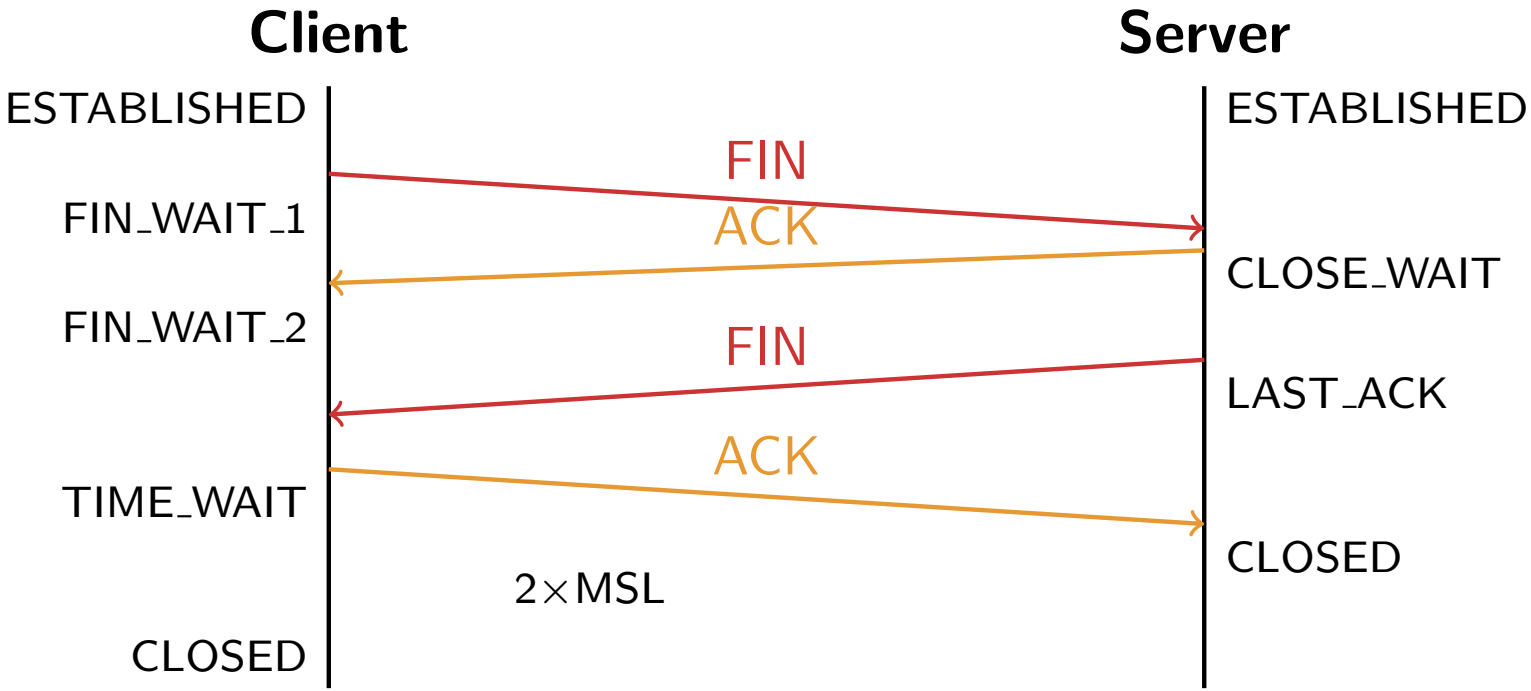
Zero Window: When rwnd=0, sender stops. Sender probes periodically to check when window opens.

TCP 3-Way Handshake



Why 3 messages? Both sides must agree on initial sequence numbers and confirm the other received them.

TCP Connection Closing



TIME_WAIT: Client waits 2×MSL (Max Segment Lifetime) to handle delayed packets.

Why Congestion Control?

The Congestion Problem

- Too many sources sending too much data too fast
- Network cannot handle the load
- Results: packet loss, long delays, wasted retransmissions

Flow Control vs Congestion Control:

	Flow Control	**Congestion Control**
Concern	Receiver capacity	Network capacity
Signal	rwnd in ACK	Packet loss / delay
Limit	Don't overflow receiver	Don't overflow network

Effective Window: $\min(\text{cwnd}, \text{rwnd})$ — limited by both!

Congestion Window (cwnd)

Congestion Window

cwnd: Sender-side limit on unACKed data, based on perceived network congestion.

Key Concepts:

- Sender maintains cwnd (not advertised, computed locally)
- Sending rate $\approx$ cwnd/RTT bytes/sec
- **ssthresh**: Slow Start threshold — divides two phases

Two Phases:

1. **Slow Start**: cwnd $<$ ssthresh $\Rightarrow$ exponential growth
2. **Congestion Avoidance**: cwnd $\geq$ ssthresh $\Rightarrow$ linear growth

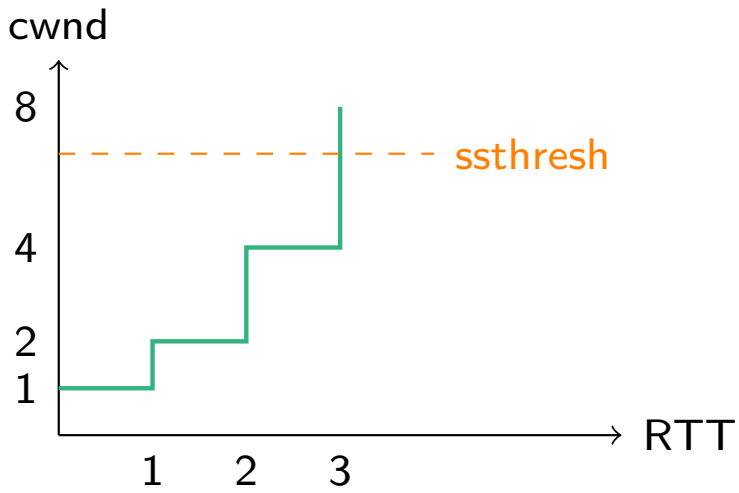
Slow Start

Slow Start Phase

When cwnd $<$ ssthresh: **cwnd doubles every RTT**

Mechanism:

- Start with cwnd = 1 MSS (or 2-10 MSS in modern TCP)
- For each ACK received: cwnd = cwnd + 1 MSS
- Effect: cwnd doubles every RTT (exponential growth)



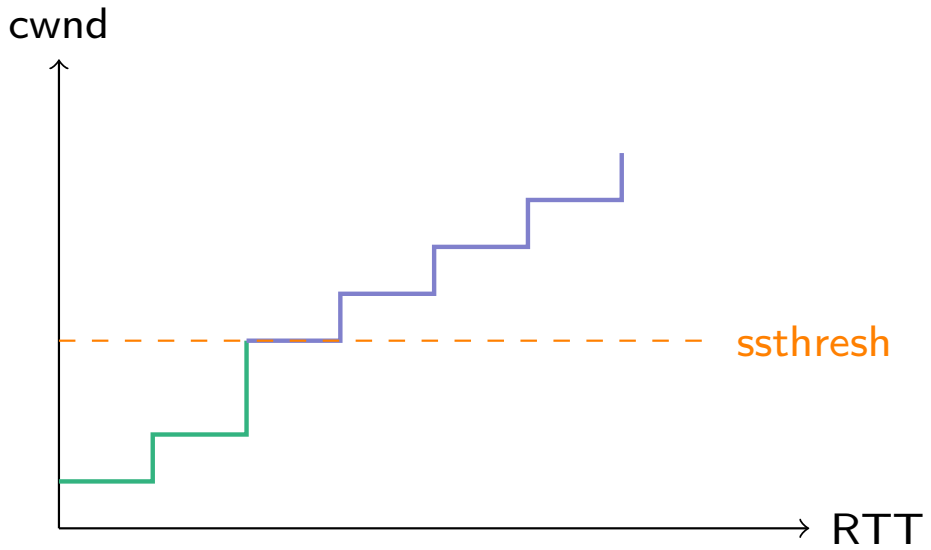
Congestion Avoidance

Congestion Avoidance Phase

When cwnd $\geq$ ssthresh: **cwnd grows by 1 MSS per RTT**

Mechanism:

- For each ACK: cwnd = cwnd + MSS$\times$(MSS/cwnd)
- Approximately: cwnd increases by 1 MSS per RTT
- Linear growth — probe for available bandwidth carefully



TCP Tahoe

TCP Tahoe: On Any Loss

On timeout OR 3 duplicate ACKs:

1. ssthresh = cwnd / 2
2. cwnd = 1 MSS
3. Enter Slow Start



Problem: Very aggressive — loses all progress on any loss event.

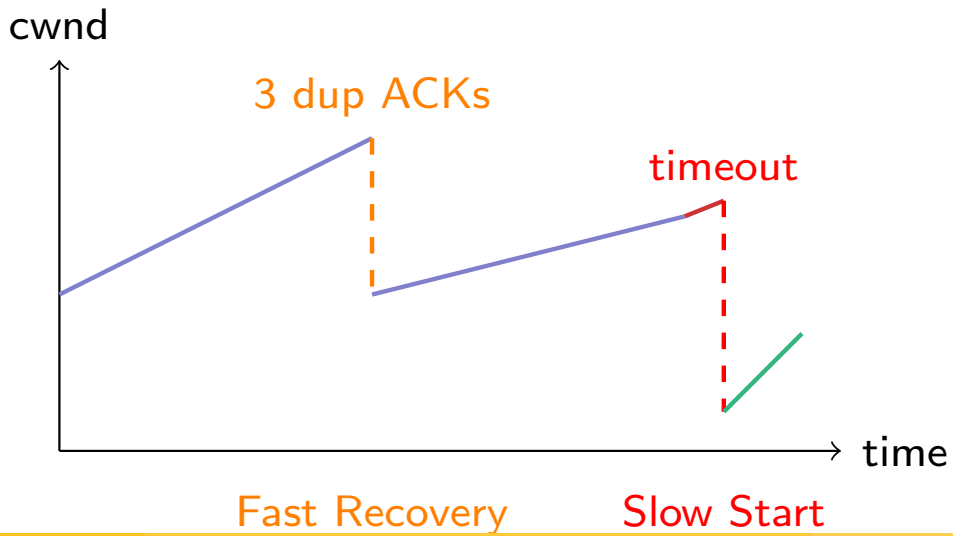
TCP Reno

TCP Reno: Distinguishes Loss Types

On timeout: Same as Tahoe (cwnd = 1, Slow Start)

On 3 duplicate ACKs (Fast Recovery):

1. ssthresh = cwnd / 2
2. cwnd = ssthresh (not 1!)
3. Enter Congestion Avoidance directly



Tahoe vs Reno Summary

Event	TCP Tahoe	TCP Reno
Timeout	cwnd = 1, SS	cwnd = 1, SS
3 Dup ACKs	cwnd = 1, SS	cwnd = ssthresh, CA
ssthresh after loss	cwnd/2	cwnd/2

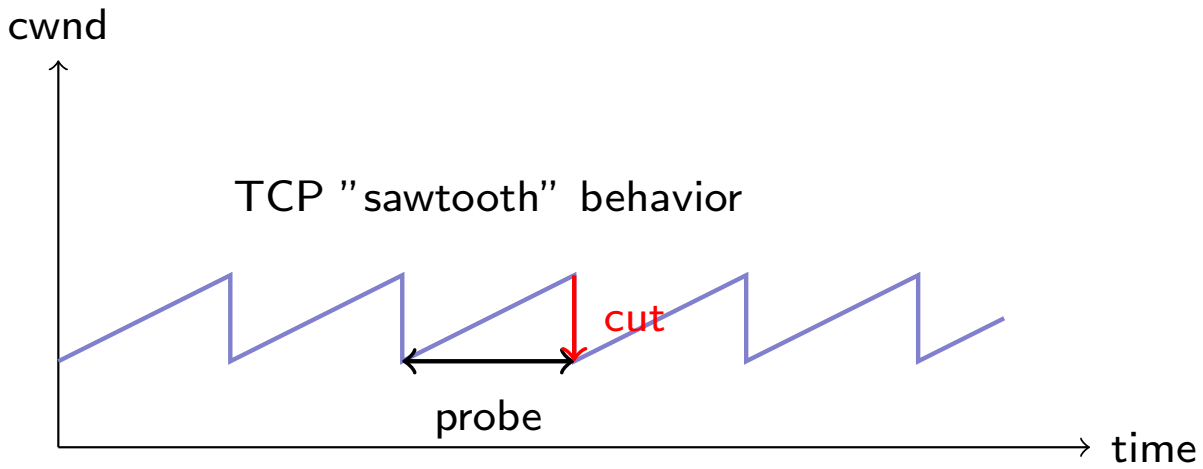
Key Insight:

- Timeout = severe congestion (no packets getting through)
- 3 Dup ACKs = mild congestion (packets still arriving)
- Reno is less aggressive on mild congestion

AIMD: Additive Increase, Multiplicative Decrease

AIMD Principle

- **Additive Increase**: Probe for bandwidth slowly (+1 MSS/RTT)
- **Multiplicative Decrease**: React to congestion quickly (cwnd/2)



Why AIMD?

Practice: Minimum Sequence Numbers

Question 1: A sliding window protocol uses a window size of $N = 5$.

(a) If using **Go-Back-N**, what is the minimum number of sequence numbers needed?

(b) If using **Selective Repeat**, what is the minimum number of sequence numbers needed?

(c) With 3-bit sequence numbers (0-7), what is the maximum window size for SR?

Practice: Minimum Sequence Numbers

Question 1: A sliding window protocol uses a window size of $N = 5$.

(a) If using **Go-Back-N**, what is the minimum number of sequence numbers needed?

(b) If using **Selective Repeat**, what is the minimum number of sequence numbers needed?

(c) With 3-bit sequence numbers (0-7), what is the maximum window size for SR?

Solution

(a) GBN: $N + 1 = 5 + 1 = 6$ sequence numbers

(b) SR: $2N = 2 \times 5 = 10$ sequence numbers

(c) SR with 8 seq #s: max window $= 8/2 = 4$

Practice: TCP Congestion Control

Question 2: TCP Reno with initial cwnd=1, ssthresh=8.

(a) What is cwnd after 4 RTTs (no loss)?

(b) After RTT 4, 3 duplicate ACKs received. What are new cwnd and ssthresh?

(c) What phase does TCP enter after the 3 dup ACKs?

Practice: TCP Congestion Control

Question 2: TCP Reno with initial cwnd=1, ssthresh=8.

(a) What is cwnd after 4 RTTs (no loss)?

(b) After RTT 4, 3 duplicate ACKs received. What are new cwnd and ssthresh?

(c) What phase does TCP enter after the 3 dup ACKs?

Solution

(a) RTT 1: cwnd=2, RTT 2: cwnd=4, RTT 3: cwnd=8 (hit ssthresh)
RTT 4: cwnd=9 (CA: +1/RTT). **cwnd = 9**

(b) ssthresh = 9/2 = 4 (floor), cwnd = 4 (Fast Recovery)

(c) **Congestion Avoidance** (Reno skips Slow Start on 3 dup ACKs)

Practice: TCP vs Tahoe vs Reno

Question 3: Compare behaviors starting with cwnd=16, ssthresh=32. A timeout occurs. What happens in:

(a) TCP Tahoe?

(b) TCP Reno?

Practice: TCP vs Tahoe vs Reno

Question 3: Compare behaviors starting with cwnd=16, ssthresh=32. A timeout occurs. What happens in:

(a) TCP Tahoe?

(b) TCP Reno?

Solution

Both Tahoe and Reno behave the same on timeout:

- ssthresh = cwnd/2 = 16/2 = 8
- cwnd = 1 MSS
- Enter Slow Start phase

Note: Tahoe and Reno only differ on 3 duplicate ACKs. On timeout, both reset to Slow Start with cwnd=1.

Practice: Congestion Window Evolution

Question 4: Trace cwnd for TCP Reno: Initial cwnd=1, ssthresh=16.

Events: Normal operation for 6 RTTs, then 3 dup ACKs at end of RTT 6.

Practice: Congestion Window Evolution

Question 4: Trace cwnd for TCP Reno: Initial cwnd=1, ssthresh=16.

Events: Normal operation for 6 RTTs, then 3 dup ACKs at end of RTT 6.

Solution

RTT	cwnd	ssthresh	Phase
1	2	16	Slow Start
2	4	16	Slow Start
3	8	16	Slow Start
4	16	16	Slow Start (hit ssthresh)
5	17	16	Congestion Avoidance
6	18	16	Congestion Avoidance
3 dup ACKs: ssthresh=9, cwnd=9, enter CA			

Practice: GBN ACK Range

Question: Consider Go-Back-N with sender window size $N = 5$. At time t, the sender base is $b = 20$. Assume no packet reordering.

- What is the minimum ACK value that may still be propagating?
- What is the maximum ACK value that may still be propagating?

Practice: GBN ACK Range

Question: Consider Go-Back-N with sender window size $N = 5$. At time t, the sender base is $b = 20$. Assume no packet reordering.

- What is the minimum ACK value that may still be propagating?
- What is the maximum ACK value that may still be propagating?

Possible ACK values: $[b - 1,\ b + N - 1] = [19,\ 24]$

Practice: Why GBN Needs $N + 1$

Question: Explain why Go-Back-N with window size N requires at least $N + 1$ sequence numbers.

Practice: Why GBN Needs $N + 1$

Question: Explain why Go-Back-N with window size N requires at least $N + 1$ sequence numbers.

With only N sequence numbers, old ACKs or retransmissions may be confused with new packets after wraparound.

Practice: Selective Repeat ACKs

Question: Consider Selective Repeat with sender window size $N = 4$. Assume no packet reordering.

- What is the maximum number of **distinct ACK values** that may be propagating in the network at the same time?

Practice: Selective Repeat ACKs

Question: Consider Selective Repeat with sender window size $N = 4$. Assume no packet reordering.

- What is the maximum number of **distinct ACK values** that may be propagating in the network at the same time?

Answer: Up to $2N = 8$ distinct ACK values.
ACKs may correspond to packets in the current receive window $[rcvbase, rcvbase + N - 1]$ as well as duplicate ACKs for packets in the previous window $[rcvbase - N, rcvbase - 1]$.

Practice: Why SR Needs $2N$

Question: Why does Selective Repeat require at least $2N$ sequence numbers?

Practice: Why SR Needs $2N$

Question: Why does Selective Repeat require at least $2N$ sequence numbers?

> Sender and receiver windows of size N must never overlap after wraparound; otherwise, old packets are indistinguishable from new ones.

Practice: TCP Loss Detection

Question: Can TCP detect packet loss before a timeout occurs? How?

Practice: TCP Loss Detection

Question: Can TCP detect packet loss before a timeout occurs? How?

Yes. TCP uses **3 duplicate ACKs** to trigger **Fast Retransmit**.

Practice: Timeout vs Duplicate ACKs

Question: Which indicates more severe congestion: a timeout or 3 duplicate ACKs?

Practice: Timeout vs Duplicate ACKs

Question: Which indicates more severe congestion: a timeout or 3 duplicate ACKs?

A timeout is more severe — it means no packets are getting through.

Practice: cwnd Growth

Question: How can you identify the TCP phase from a cwnd graph?

Practice: cwnd Growth

Question: How can you identify the TCP phase from a cwnd graph?

Exponential growth $\rightarrow$ Slow Start
Linear growth $\rightarrow$ Congestion Avoidance

Practice: Reno vs Tahoe

Question: What happens after 3 duplicate ACKs in:

- TCP Tahoe?
- TCP Reno?

Practice: Reno vs Tahoe

Question: What happens after 3 duplicate ACKs in:

- TCP Tahoe?
- TCP Reno?

Tahoe: $cwnd = 1$, Slow Start
Reno: $cwnd = ssthresh$, Congestion Avoidance

Practice: Effective Window

Question: What limits the amount of in-flight data in TCP?

Practice: Effective Window

Question: What limits the amount of in-flight data in TCP?

$\min(cwnd, rwnd)$

Practice: Conceptual TCP Question

Question: Why does TCP use cumulative ACKs instead of per-packet ACKs?

Practice: Conceptual TCP Question

Question: Why does TCP use cumulative ACKs instead of per-packet ACKs?

> They reduce ACK traffic and simplify sender state while still enabling fast loss detection.